

FDS0: Foundations of FDS

Seminar 10: $\text{\LaTeX}$ Basics

Redstone, JerrySu

May 2026

The Field of Dream Studio

Table of content

1. Why $\text{\LaTeX}$?
2. Install $\text{\LaTeX}$
3. Hello World in $\text{\LaTeX}$
4. Sections and Table of Contents
5. Equations
6. Typesetting Code

1

Why $\text{\LaTeX}$?

Why $\text{\LaTeX}$, after Markdown?

You already know Markdown (Seminar 05). $\text{\LaTeX}$ answers a different question.

Markdown is intentionally light (no math, no theorems, no cross-references) and fragmented across incompatible flavours (CommonMark, GFM, MultiMarkdown, ...). $\text{\LaTeX}$ is for documents whose structure, math, and cross-references must hold up under review.

- **Structural semantics.** `\section`, `\theorem`, `\proof` carry meaning; numbering follows automatically.
- **Math.** Inline and display math without ad-hoc images: $\int_0^1 x^2 dx$, $\sum$, $\forall$.
- **Reviewable source.** Plain-text diffs review like any other file in Git.
- **Reproducible PDFs.** The same `.tex` compiles to the same PDF on any reviewer's machine.
- **Cross-references.** `\label` and `\ref` stay correct when sections move.

Where $\text{\LaTeX}$ Is Required

Knowing $\text{\LaTeX}$ is not optional in three places.

- **FDS presentations.** All FDS decks, including this one, are built from the FDS $\text{\LaTeX}$ template (Seminar 11).
- **Academic submission.** Most maths, CS, and physics journals and conferences require $\text{\LaTeX}$ source for the camera-ready PDF.
- **AI-assisted writing.** $\text{\LaTeX}$ is plain text and well-specified, so AI agents (Seminars 12 and 13) produce reliable $\text{\LaTeX}$ sources; you review the diff exactly as in Seminar 13.



2

Install $\text{\LaTeX}$



Hello World in $\text{\LaTeX}$

Minimal Code

After you installing the $\text{\LaTeX}$ environment, create a file called `hello_world.tex`. And paste the following code into that file.

```
\documentclass[12pt]{article}  
\begin{document}  
Hello World  
\end{document}
```


Compile the Code

Click the Compile Button

After pasting the code, you should see the compile button on the screen.

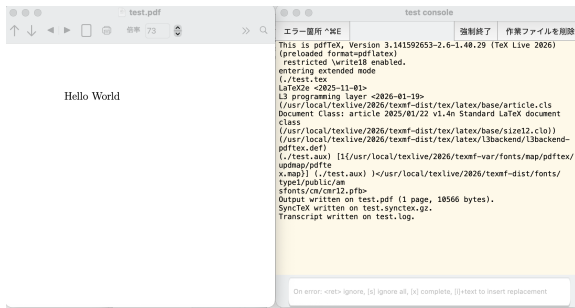
Make sure that the compile you use is pdfLaTeX or LaTeX. (The underlined part)



Compile the Code (Cont'd)

Product of Compilation

1. **Compile Logs** on the right hand side, showing the warnings or errors.
2. **Compiled PDF** on the left hand side, showing "Hello World" on the paper.



Preamble

The **preamble** is everything before `\begin{document}`. In our code, it is

```
\documentclass[12pt]{article}
```

This line tells LaTeX what kind of document you want to create.

documentclass command

We use the `documentclass` command to tell L^AT_EX what kind of document we want to create. the `[12pt]` tells it that the normal font should be 12pt, and `{article}` tells it that our document is an article.

You may change the `[12pt]` into other sizes, eg, `[10pt]` to see the difference in size for the texts.

The Document Body

The **body** is the part between `\begin{document}` and `\end{document}`

Where this tells LaTeX: the actual visible content is between that. Anything in between this will appear in the final PDF, unless it is a command or comment.

Hello World is just a normal text, so it get rendered on our PDF normally

Comments

The % character starts a comment.

Anything after a % on the same line is skipped by L^AT_EX. Comments are useful for leaving notes for yourself, or for temporarily disabling a line without deleting it.

```
\documentclass[12pt]{article}
\begin{document}
Hello World    % rendered text, this note is not
% this whole line is skipped by the compiler
\end{document}
```

To print a literal percent sign, escape it as `\%`.

A bare % is always read as the start of a comment, so it cannot reach the PDF on its own.

Fonts

Style commands wrap the text they affect.

The braces mark exactly which text each command applies to, and commands can be nested to combine effects.

- `\textbf{...}` for **bold**
- `\textit{...}` for *italic*
- `\texttt{...}` for typewriter
- `\textbf{\textit{...}}` for **bold italic**

This is normal text.

This text is bold.

This text is italic.

This text is in typewriter font.

This text is bold and italic.

Font Sizes

Size commands apply from where they appear, inside their group.

Wrap a command in braces so its effect ends at the closing brace. The document default size comes from `\documentclass`, for example `[12pt]`.

- `\small`
- `\normalsize`
- `\large`
- `\Large`
- `\LARGE`
- `\huge`

This phrase is small.

This phrase is at the normal size.

This phrase is large.

This phrase is Large.

This phrase is LARGE.

This phrase is huge.



Sections and Table of Contents

Section Headings

Sectioning commands organize a document into a hierarchy.

$\text{\LaTeX}$ numbers them automatically and adds them to the table of contents. The braces hold the heading text.

- `\section{...}` (top level: 1, 2, 3)
- `\subsection{...}` (second level: 1.1, 1.2)
- `\subsubsection{...}` (third level: 1.1.1)

All sectioning commands live in the body.

You do not declare anything in the preamble. Write the command on its own line where the heading should appear, and $\text{\LaTeX}$ takes care of the numbering and the spacing.

Numbered and Starred Forms

The default form is numbered.

`\section{Introduction}` produces a heading like **1 Introduction**, and the number updates automatically as sections are added or removed.

The starred form is unnumbered.

`\section*{Acknowledgements}` produces just **Acknowledgements**, with no number. The same applies to `\subsection*{...}` and `\subsubsection*{...}`.

Starred headings are skipped by the table of contents.

Useful for an abstract, an acknowledgements block, or any heading that should not be counted as a numbered part of the document.

Worked Example: Source

Preamble and body, revisited.

The preamble (above `\begin{document}`) declares the document class. The body (between `\begin{document}` and `\end{document}`) holds everything that prints, including all sectioning commands.

```
\documentclass[12pt]{article}      % preamble: article class
\begin{document}                  % body opens
\tableofcontents                  % auto-built TOC
\section{Introduction}             % numbered: 1
Top-level numbered section.
\subsection{Background}           % numbered: 1.1
\subsubsection{Details}           % numbered: 1.1.1
\section{Methods}                 % numbered: 2
\section*{Acknowledgements}       % unnumbered, not in TOC
\end{document}                   % body closes
```

Walkthrough, Line by Line

Preamble.

- `\documentclass[12pt]{article}`: choose the article class at base 12pt; this class provides the sectioning commands.

Body frame.

- `\begin{document}`: open the body.
- `\tableofcontents`: reserve a slot for the auto-built TOC; populated on a second compile pass.
- `\end{document}`: close the body.

Walkthrough, Line by Line (Cont'd)

Each heading is followed by the paragraphs that belong to it.

The sectioning commands appear in the body in order, and $\text{\LaTeX}$ numbers them automatically.

- `\section{Introduction}`: prints **1 Introduction**.
- `\subsection{Background}`: prints **1.1 Background**.
- `\subsubsection{Details}`: prints **1.1.1 Details**.
- `\section{Methods}`: prints **2 Methods**.
- `\section*{Acknowledgements}`: unnumbered, not in TOC.

Compiled Output

The body shows the headings; the TOC lists only the numbered ones.

The starred *Acknowledgements* appears in the body but is absent from the table of contents. The TOC is populated only on a second compile pass.

Body

1 Introduction

Top-level numbered section.

1.1 Background

Numbered subsection.

1.1.1 Details

Numbered sub-subsection.

2 Methods

Another top-level numbered section.

Acknowledgements

A starred section: no number, not in the table of contents.

Table of Contents

Contents

1 Introduction	1
1.1 Background	1
1.1.1 Details	1
2 Methods	1



5

Equations

Inline Math

Inline math sits inside a sentence.

Wrap the formula in dollar signs: $\$ \dots \$$. $\text{\LaTeX}$ keeps the math at the height of the surrounding text so the formula flows with the line.

Example.

The solution to $\$x+3=5\$$ is $\$x=2\$$. renders as: the solution to $x + 3 = 5$ is $x = 2$.

Big constructs get squeezed inline.

The same integral, inline: $\int_0^1 x^2 dx = \frac{1}{3}$. As display:

$$\int_0^1 x^2 dx = \frac{1}{3}.$$

Inline compresses the limits and the fraction so line height stays normal; display gives them their full vertical room.

Superscripts and Subscripts

Use `^` for superscript and `_` for subscript.

A single character follows the operator directly; multi-character expressions need braces.

- `x^2` gives x^2 .
- `x_i` gives x_i .
- `x_i^2` gives x_i^2 (both at once).
- `a_{i+1}^{n-1}` gives a_{i+1}^{n-1} .

Without braces, only the first character is taken.

`x^10` produces x^10 (only the 1 is raised). Write `x^{10}` to get x^{10} .

Greek Alphabet

Greek letters are commands in math mode.

Lowercase: `\alpha` gives α , and so on. Uppercase that looks identical to a Latin capital (A, B, E, Z, ...) is typed as the Latin letter; the eleven distinct uppercase forms have their own commands.

α <code>\alpha</code>	β <code>\beta</code>	γ <code>\gamma</code>	δ <code>\delta</code>
ϵ <code>\epsilon</code>	ζ <code>\zeta</code>	η <code>\eta</code>	θ <code>\theta</code>
ι <code>\iota</code>	κ <code>\kappa</code>	λ <code>\lambda</code>	μ <code>\mu</code>
ν <code>\nu</code>	ξ <code>\xi</code>	o (<i>plain o</i>) <code>\pi</code>	π <code>\pi</code>
ρ <code>\rho</code>	σ <code>\sigma</code>	τ <code>\tau</code>	υ <code>\upsilon</code>
ϕ <code>\phi</code>	χ <code>\chi</code>	ψ <code>\psi</code>	ω <code>\omega</code>

Uppercase commands. `\Gamma` Γ , `\Delta` Δ , `\Theta` Θ , `\Lambda` Λ , `\Xi` Ξ , `\Pi` Π , `\Sigma` Σ , `\Upsilon` Υ , `\Phi` Φ , `\Psi` Ψ , `\Omega` Ω .

Display Math, Unnumbered

Display math sits alone on its own line and is centered.

Use `\[ ... \]` in modern L^AT_EX. The old plain-TeX form `$$ ... $$` still works but is discouraged because it interacts badly with some spacing rules.

Example.

`\[ x^2 + y^2 = r^2 \]` renders as

$$x^2 + y^2 = r^2$$

Big constructs get their full vertical room.

$$\int_0^1 x^2 dx = \frac{1}{3}, \quad \sum_{n=1}^{10} n = 55$$

Limits sit above and below; fractions stay tall.

The equation Environment

Use equation for a single numbered display line.

$\LaTeX$ assigns the number automatically and increments across the document.

$$E = mc^2 \tag{1}$$

Add `\label{...}` to refer back later.

```
\begin{equation} \label{eq:mass-energy}
```

```
  E = mc^2
```

```
\end{equation}
```

```
... see equation~\ref{eq:mass-energy}.
```

The align Environment: & and \\

align stacks lines, all numbered. End each line with \\; place & just before the symbol you want lined up.

```
\begin{align}
  x + 3 &= 5 \\
  x &= 2 \\
\end{align}
```

renders as

$$x + 3 = 5 \tag{1}$$

$$x = 2 \tag{2}$$

Anchor on any relation.

Use &\leq, &\to, or any other symbol as the column.

Starred Forms: `equation*` and `align*`

Add a `*` to suppress the equation number.

`equation*` gives an unnumbered display line; `align*` gives unnumbered aligned lines. Use the starred form for derivation steps and expository math; number equations you may cite later.

Numbered (`align`):

$$x + 3 = 5 \tag{1}$$

$$x = 2 \tag{2}$$

Unnumbered (`align*`):

$$x + 3 = 5$$

$$x = 2$$

Useful Commands: Middle-School Maths

Most math symbols have their own command.

Type `\times` for $\times$, `\leq` for $\leq$, `\sqrt{x}` for $\sqrt{x}$. The backslash plus name is the universal pattern.

Beyond the basics, add `\usepackage{amssymb}`.

It provides `\mathbb`, `\sqsubseteq`, `\triangleq`, and many other symbols used on the next two slides.

$\times$ <code>\times</code>	$\div$ <code>\div</code>	$\pm$ <code>\pm</code>	$\cdot$ <code>\cdot</code>
$\sqrt{x}$ <code>\sqrt{x}</code>	$\frac{a}{b}$ <code>\frac{a}{b}</code>	$\angle$ <code>\angle</code>	$\triangle$ <code>\triangle</code>
$\perp$ <code>\perp</code>	$\parallel$ <code>\parallel</code>	$\cong$ <code>\cong</code>	$\sim$ <code>\sim</code>
$\log$ <code>\log</code>	$\ln$ <code>\ln</code>	$\sin$ <code>\sin</code>	$\cos$ <code>\cos</code>

For degrees, write `90^\circ` to get 90° . Tangent follows the pattern: `\tan`.

Useful Commands: University Maths and PL Theory

Top rows: university maths. Bottom rows: PL theory. The double brackets `\llbracket`, `\rrbracket` need `\usepackage{stmaryrd}`.

$\forall$ <code>\forall</code>	$\exists$ <code>\exists</code>	$\in$ <code>\in</code>	$\subseteq$ <code>\subseteq</code>
$\cup$ <code>\cup</code>	$\cap$ <code>\cap</code>	$\emptyset$ <code>\emptyset</code>	∞ <code>\infty</code>
$\leq$ <code>\leq</code>	$\geq$ <code>\geq</code>	$\neq$ <code>\neq</code>	$\equiv$ <code>\equiv</code>
$\Rightarrow$ <code>\Rightarrow</code>	$\Leftrightarrow$ <code>\Leftrightarrow</code>	$\rightarrow$ <code>\rightarrow</code>	$\mapsto$ <code>\mapsto</code>
$\sum$ <code>\sum</code>	$\prod$ <code>\prod</code>	$\int$ <code>\int</code>	$\mathbb{R}$ <code>\mathbb{R}</code>
$\vdash$ <code>\vdash</code>	$\models$ <code>\models</code>	$\perp$ <code>\perp</code>	$\top$ <code>\top</code>
$\sqsubseteq$ <code>\sqsubseteq</code>	$\trianglelefteq$ <code>\trianglelefteq</code>	$\langle$ <code>\langle</code>	$\rangle$ <code>\rangle</code>
$\llbracket$ <code>\llbracket</code>	$\rrbracket$ <code>\rrbracket</code>	$\Downarrow$ <code>\Downarrow</code>	$\rightsquigarrow$ <code>\rightsquigarrow</code>

Worked Example: $x + 3 = 5$, $x = 2$

Inline (`$ ... $`): substitute into prose, like $x + 3 = 5$ so $x = 2$.

Display, unnumbered (`\[ ... \]`):

$$x + 3 = 5 \quad \Rightarrow \quad x = 2$$

Display, numbered (equation):

$$x + 3 = 5 \tag{1}$$

Multi-line, aligned (`align`):

$$x + 3 = 5 \tag{2}$$

$$x = 2 \tag{3}$$



6 Typesetting Code

verbatim for Plain Code

The verbatim environment prints everything literally.

Nothing inside it is interpreted as a $\text{\LaTeX}$ command. Wrap your code in `\begin{verbatim}` and `\end{verbatim}`.

```
def hello():
    print("Hello World")
for i in range(3):
    hello()
```

Inline form: `\verb|...|`.

Use any character as the delimiter (`|`, `!`, `+`, ...) so long as it does not appear in the code itself.

lstlisting for Pretty Code

The listings package adds language-aware rendering.

Load it with `\usepackage{listings}`. Use the `lstlisting` environment like `verbatim`, with optional language and style settings in square brackets.

```
def hello(name):
    return f"Hello , -{name}!"
```

```
print( hello("World" ))
```

Options live in square brackets after the begin tag.

`[language=Python]` marks keywords distinctly; `[numbers=left]` adds line numbers; `[caption=My code]` adds a caption. Set deck-wide defaults with `\lstset{...}` in the preamble.

Recap

What we covered today.

- **Hello World:** preamble, body, comments, fonts, font sizes.
- **Sections:** `\section`, `\subsection`, starred forms, `\tableofcontents`.
- **Equations:** inline, display, `equation`, `align`, `align*`, the `&` alignment marker.
- **Useful symbols:** backslash-name pattern, Greek alphabet, middle-school maths, university maths, PL theory.
- **Code:** `verbatim`, `\verb`, `lstlisting`.